

ĐẠI HỌC QUỐC GIA, THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



CẤU TRÚC RỜI RẠC (CO1007)

BÁO CÁO BÀI TẬP LỚN

Thuật toán A^*

Sinh viên: Hoàng Đức Tài
MSSV: 2510219

TP Hồ Chí Minh, Tháng 5 Năm 2026



Mục lục

1 Task 1: Degrees of Separation in a Social Network - Giải thích cách tính hàm Heuristic và chạy một ví dụ cụ thể	2
1.1 Phương pháp tính toán hàm Heuristic	2
1.2 Ví dụ	3
1.2.1 Ma trận kề ban đầu và giá trị Heuristic	3
1.2.2 Tiến trình duyệt của thuật toán A^*	3
1.2.3 Output:	4
2 Task 2: Drone Delivery in 2D Space with Three Heuristics - So sánh và đánh giá ưu điểm của các hàm Heuristic	4
2.1 Hàm khoảng cách Manhattan	4
2.2 Hàm khoảng cách Euclidean	4
2.3 Hàm khoảng cách Chebyshev	5
3 Task 3: Warehouse Robot Navigation with Obstacles - So sánh 2 hàm Heuristic qua một ví dụ bản đồ lưới của một kho nhỏ	5
3.1 Hàm khoảng cách Manhattan	5
3.2 Hàm khoảng cách Chebyshev	6
3.3 So sánh và Đánh giá tổng quan	6
4 Task 4: Evacuation Route - Quy trình chuyển đổi từ bản đồ lưới sang ma trận trọng số và so sánh các hàm Heuristic qua một ví dụ	7
4.1 Quy trình chuyển đổi từ lưới sơ đồ sang ma trận trọng số	7
4.2 Ví dụ	7
4.2.1 Hàm khoảng cách Manhattan	7
4.2.2 Hàm khoảng cách Chebyshev	8
4.3 So sánh và Đánh giá tổng quan	8

1 Task 1: Degrees of Separation in a Social Network - Giải thích cách tính hàm Heuristic và chạy một ví dụ cụ thể

1.1 Phương pháp tính toán hàm Heuristic

Hàm Heuristic được xây dựng dựa trên thuật toán BFS kết hợp với cơ chế lưu trữ tĩnh. Giải thuật thực hiện duyệt BFS từ Node đích để tính toán và thiết lập bảng khoảng cách ngắn nhất từ mọi nút về đích trên đồ thị. Cơ chế bộ nhớ đệm này giúp loại bỏ hoàn toàn việc tính toán lặp lại dư thừa giữa các Node, tối ưu hóa hiệu năng cho các lượt gọi sau:

```
1 double h(double adjMatrix[100][100], int curr, int goal)
2 {
3     static std::vector<double> dist(100, 1e9);
4     static int lastGoal = -1;
5
6     if (goal != lastGoal)
7     {
8         std::fill(dist.begin(), dist.end(), 1e9);
9         dist[goal] = 0;
10
11        std::vector<int> que;
12        que.push_back(goal);
13        int head = 0;
14
15        while (head < que.size())
16        {
17            int c = que[head++];
18            for (int n = 0; n < 100; n++)
19                if (adjMatrix[n][c] > 0 && dist[n] > dist[c] + 1)
20                {
21                    dist[n] = dist[c] + 1;
22                    que.push_back(n);
23                }
24        }
25        lastGoal = goal;
26    }
27
28    return dist[curr];
29 }
```

Quy trình tính toán chi tiết của hàm Heuristic được cấu trúc qua bốn giai đoạn cốt lõi sau:

- **Giai đoạn 1: Khởi tạo**

- Từ khóa `static` giúp mảng `dist` và biến `lastGoal` duy trì vùng nhớ cố định suốt vòng đời chương trình để làm bộ nhớ đệm.
- Mảng `dist` khởi tạo giá trị vô cùng (10^9) đại diện cho trạng thái các Node chưa được thuật toán duyệt tới để tính khoảng cách.
- Biến `lastGoal` khởi tạo bằng -1 nhằm bắt buộc hệ thống phải duyệt BFS tính toán bảng khoảng cách từ đầu ở lượt gọi đầu tiên.

- **Giai đoạn 2: Điều kiện kích hoạt và làm mới trạng thái**

- Điều kiện `if (goal != lastGoal)` giúp giải thuật BFS chỉ kích hoạt duyệt đồ thị lại từ đầu khi Node đích thực sự thay đổi hoặc ở lượt gọi đầu tiên.
- Nếu Node đích trùng với lượt tìm kiếm trước, bỏ qua chạy lại giải thuật BFS và dùng kết quả đã tính trước đó.

- **Giai đoạn 3: Chạy thuật toán BFS để tính khoảng cách ngắn nhất**
 - Sử dụng vòng lặp **while** và hàng đợi **que** để thực hiện giải thuật BFS duyệt dần từ Node đích ra toàn bộ đồ thị nhằm tính khoảng cách ngắn nhất đến các Node.
 - Khi duyệt qua các Node lân cận hợp lệ, hệ thống sẽ cập nhật lại khoảng cách ngắn nhất vào mảng **dist** nếu tìm thấy đường đi mới tối ưu hơn.
- **Giai đoạn 4: Trả kết quả truy xuất**
 - Sau khi kết thúc quá trình tính toán (hoặc đọc trực tiếp từ kết quả lần tính trước), hàm trả về kết quả bằng lệnh **return dist[curr]**.
 - Nhờ cơ chế này, ngoại trừ Node đầu tiên chịu chi phí duyệt đồ thị ban đầu, tất cả các lần gọi hàm tiếp theo cho các Node khác trong luồng chạy của A^* đều đạt tốc độ lý tưởng là $O(1)$.

1.2 Ví dụ

Giả sử đồ thị mạng xã hội gồm có 5 người (được đánh chỉ số từ 0 đến 4). Hệ thống cần tìm kiếm đường đi ngắn nhất từ **Person 0 (Start)** đến **Person 4 (Goal)**.

1.2.1 Ma trận kề ban đầu và giá trị Heuristic

Ma trận kề **adjMatrix** biểu diễn mối quan hệ kết nối trực tiếp (trọng số 1) giữa các Node:

$$\text{adjMatrix} = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

Tại lượt gọi đầu tiên với đích là Node 4, hàm chạy toàn bộ đồ thị một lần và lưu trữ bảng tính: $h(4) = 0$, $h(3) = 1$, $h(1) = 2$, $h(2) = 2$, $h(0) = 3$. Các lượt gọi tiếp theo của thuật toán A^* chỉ tốn chi phí $O(1)$ để truy xuất trực tiếp từ bộ nhớ đệm này.

1.2.2 Tiến trình duyệt của thuật toán A^*

Thuật toán quản lý trạng thái các Node qua **Open List** (chờ duyệt, ưu tiên giá trị f nhỏ nhất) và **Closed List** (đã duyệt xong).

Bước duyệt	Node hiện tại	Open List (Node, g, h, f)	Closed List
Khởi tạo	-	(0, 0, 3, 3)	$\emptyset$
Bước 1	0	(1, 1, 2, 3), (2, 1, 2, 3)	{0}
Bước 2	1	(2, 1, 2, 3), (3, 2, 1, 3)	{0, 1}
Bước 3	2	(3, 2, 1, 3)	{0, 1, 2}
Bước 4	3	(4, 3, 0, 3)	{0, 1, 2, 3}
Bước 5	4	$\emptyset$ (Đã đạt đích)	{0, 1, 2, 3, 4}

1.2.3 Output:

- Node: 0 | $f : 3.0$ | $g : 0.0$ | $h : 3.0$
- Node: 1 | $f : 3.0$ | $g : 1.0$ | $h : 2.0$
- Node: 3 | $f : 3.0$ | $g : 2.0$ | $h : 1.0$
- Node: 4 | $f : 3.0$ | $g : 3.0$ | $h : 0.0$

Đường đi tối ưu hoàn chỉnh tìm được là: $0 \rightarrow 1 \rightarrow 3 \rightarrow 4$.

2 Task 2: Drone Delivery in 2D Space with Three Heuristics - So sánh và đánh giá ưu điểm của các hàm Heuristic

Để làm rõ ưu thế đặc trưng và tính chấp nhận được của từng hàm Heuristic, chúng ta tiến hành khảo sát hành vi duyệt giải thuật thông qua các cấu hình môi trường khác nhau.

2.1 Hàm khoảng cách Manhattan

Bối cảnh: Không gian di chuyển của drone bị ràng buộc nghiêm ngặt theo các trục tọa độ vuông góc (ví dụ như hệ thống hành lang đô thị). Giả sử hệ thống gồm 3 điểm với bảng tọa độ: $A(0,0)$, $B(2,0)$, $C(2,2)$. Cần tìm đường từ A đến C . Do cấu trúc địa hình cấm bay chéo, ma trận trọng số chỉ thiết lập kết nối dọc theo trục: $\text{edge}(A,B) = 2$, $\text{edge}(B,C) = 2$. Đường nối thẳng AC không tồn tại. Chi phí thực tế tối ưu là $h^*(A) = 4$.

Bước duyệt	Node hiện tại	Open List (Node, g, h, f)	Closed List
Khởi tạo	-	$(A, 0, 4, 4)$	$\emptyset$
Bước 1	A	$(B, 2, 2, 4)$	$\{A\}$
Bước 2	B	$(C, 4, 0, 4)$	$\{A, B\}$
Bước 3	C	$\emptyset$ (Đã đạt đích)	$\{A, B, C\}$

Phân tích lợi thế và Tính "chấp nhận được": Vì môi trường bắt buộc di chuyển vuông góc, khoảng cách Manhattan đạt trạng thái lý tưởng khi khớp chính xác 100% với thực địa ($h(A) = h^*(A) = 4$). Hệ thức $h \leq h^*$ được thỏa mãn, giúp hàm đạt độ chính xác tuyệt đối, định hướng thuật toán A^* đi thẳng một mạch về đích mà không mở rộng bất kỳ nhánh thừa nào.

2.2 Hàm khoảng cách Euclidean

Bối cảnh: Không gian bay hoàn toàn tự do, không có chướng ngại vật, cho phép drone di chuyển theo đường chim bay thẳng góc bất kỳ. Xét cấu trúc 3 điểm tọa độ: $A(0,0)$, $B(3,1)$, $C(3,4)$. Điểm xuất phát là A , đích là C . Ma trận kết nối thực tế cho phép bay thẳng đường chéo: $\text{edge}(A,B) = \sqrt{10} \approx 3.16$, $\text{edge}(B,C) = 3$, $\text{edge}(A,C) = 5$. Chi phí thực tế tối ưu là $h^*(A) = 5$.

Bước duyệt	Node hiện tại	Open List (Node, g, h, f)	Closed List
Khởi tạo	-	$(A, 0, 5, 5)$	$\emptyset$
Bước 1	A	$(C, 5, 0, 5)$, $(B, 3.16, 3, 6.16)$	$\{A\}$
Bước 2	C	$(B, 3.16, 3, 6.16)$ (Đã đạt đích)	$\{A, C\}$

Phân tích lợi thế và "Tính chấp nhận được": Trong không gian tự do, đường ngắn nhất luôn là đường thẳng tuyến tính. Hàm Euclidean cho ra kết quả bằng đúng chi phí thực tế ($h(A) = h^*(A) = 5$), đảm bảo tính chấp nhận được. Ngược lại, nếu áp dụng hàm Manhattan tại kịch bản này, giá trị ước lượng sẽ là $3 + 4 = 7$. Do $7 > 5$, hàm Manhattan rơi vào trạng thái đánh giá vượt quá, vi phạm tính chấp nhận được và có thể khiến giải thuật A^* không còn đảm bảo tìm được lời giải tối ưu trong các bản đồ phức tạp hơn.

2.3 Hàm khoảng cách Chebyshev

Bối cảnh: Môi trường lưới cho phép di chuyển dọc, ngang, và chéo với chi phí đồng nhất bằng 1 đơn vị hành trình cho mỗi bước đi (tương tự như quy tắc di chuyển của quân Vua trên bàn cờ). Xét hệ thống 3 điểm tọa độ: $A(0,0)$, $B(2,0)$, $C(2,2)$. Cần đi từ A đến C . Trọng số di chuyển trên ma trận đồ thị được thiết lập: $\text{edge}(A,B) = 2$, $\text{edge}(B,C) = 2$, $\text{edge}(A,C) = 2$ (do đường chéo bằng chi phí đi thẳng). Chi phí thực tế tối ưu là $h^*(A) = 2$.

Bước duyệt	Node hiện tại	Open List (Node, g, h, f)	Closed List
Khởi tạo	-	$(A, 0, 2, 2)$	$\emptyset$
Bước 1	A	$(C, 2, 0, 2)$, $(B, 2, 2, 4)$	$\{A\}$
Bước 2	C	$(B, 2, 2, 4)$ (Đã đạt đích)	$\{A, C\}$

Phân tích lợi thế và Tính "chấp nhận được": Khi bước đi chéo không chịu phạt chi phí, số bước hành trình thực tế chỉ phụ thuộc vào trục có độ lệch biên độ lớn nhất $\max(\Delta x, \Delta y)$. Hàm Chebyshev mô phỏng hoàn hảo luật này nên cho ra ước lượng bằng đúng thực tế ($h(A) = h^*(A) = 2$), thỏa mãn tính chấp nhận được. Nếu sử dụng hàm Euclidean ($h = \sqrt{8} \approx 2.83$) hoặc Manhattan ($h = 4$) tại môi trường này, cả hai hàm đều trả về giá trị lớn hơn 2.0, dẫn đến việc vi phạm nghiêm trọng tính chấp nhận được của giải thuật.

3 Task 3: Warehouse Robot Navigation with Obstacles - So sánh 2 hàm Heuristic qua một ví dụ bản đồ lưới của một kho nhỏ

Bối cảnh: Xét một lưới nhà kho không có vật cản kích thước 3×3 . Robot cần tìm đường di chuyển từ ô xuất phát $A(0,0)$ đến ô đích $C(2,2)$. Hệ thống áp dụng quy tắc di chuyển 8 hướng với mức chi phí: thẳng = 1.0, chéo = 1.5. Lộ trình thực tế ngắn nhất là đi chéo trực tiếp từ $(0,0) \rightarrow (1,1) \rightarrow (2,2)$ với tổng chi phí thực tế tối ưu $g^*(C) = 1.5 + 1.5 = 3.0$.

3.1 Hàm khoảng cách Manhattan

Khi cấu hình thuật toán chạy với hàm ước lượng Heuristic Manhattan, tiến trình mở rộng nhãn trạng thái và thông tin lưu vết tại mỗi bước được thể hiện như sau:

Bước duyệt	Node hiện tại	Open List (Node, g, h, f)	Closed List
Khởi tạo	-	$((0,0), 0, 4, 4)$	$\emptyset$
Bước 1	$(0,0)$	$((1,1), 1.5, 2, 3.5)$, $((1,0), 1, 3, 4)$, $((0,1), 1, 3, 4)$	$\{(0,0)\}$
Bước 2	$(1,1)$	$((2,2), 3.0, 0, 3.0)$, $((2,1), 2.5, 1, 3.5)$, ...	$\{(0,0), (1,1)\}$
Bước 3	$(2,2)$	$\emptyset$ (Đã đạt đích)	$\{(0,0), (1,1), (2,2)\}$

Output:

- Node 1: Down-Right | $f : 3.5$ | $g : 1.5$ | $h : 2.0$
- Node 2: Down-Right | $f : 3.0$ | $g : 3.0$ | $h : 0.0$

3.2 Hàm khoảng cách Chebyshev

Khảo sát tiến trình duyệt của thuật toán trên cùng kích bản lưới và luật di chuyển 8 hướng, nhưng thay đổi cấu hình sang hàm ước lượng Chebyshev:

Bước duyệt	Node hiện tại	Open List (Node, g, h, f)	Closed List
Khởi tạo	-	$((0, 0), 0, 2, 2)$	$\emptyset$
Bước 1	$(0, 0)$	$((1, 1), 1.5, 1, 2.5), ((1, 0), 1, 2, 3), ((0, 1), 1, 2, 3)$	$\{(0, 0)\}$
Bước 2	$(1, 1)$	$((2, 2), 3.0, 0, 3.0), ((1, 0), 1, 2, 3), ((0, 1), 1, 2, 3)$	$\{(0, 0), (1, 1)\}$
Bước 3	$(2, 2)$	$\emptyset$ (Đã đạt đích)	$\{(0, 0), (1, 1), (2, 2)\}$

Output:

- Node 1: Down-Right | $f : 2.5$ | $g : 1.5$ | $h : 1.0$
- Node 2 Down-Right | $f : 3.0$ | $g : 3.0$ | $h : 0.0$

3.3 So sánh và Đánh giá tổng quan

Từ việc khảo sát thực nghiệm trên cùng một ví dụ hệ thống với quy tắc phạt chi phí đường chéo bằng 1.5, ta rút ra nhận xét mang tính chất định lượng như sau:

- Về số lượng ô phải duyệt: Trong kích bản lưới quy mô nhỏ và không có vật cản này, cả hai hàm Heuristic đều định hướng rất tốt, giúp thuật toán A^* hội tụ nhanh chóng về đích chỉ sau đúng 3 bước duyệt mà không bị loang sang các Node lân cận không tối ưu.
- Về tính chất "chấp nhận được":
 - Hàm Chebyshev: Đưa ra giá trị ước lượng tại điểm nguồn là $h = 2$. Vì $2.0 \leq 3.0$ (chi phí thực tế tối ưu), hàm Chebyshev luôn nhỏ hơn hoặc bằng chi phí thực tế, hoàn toàn thỏa mãn tính chất chấp nhận được. Do đó, giải thuật A^* sử dụng hàm Chebyshev luôn tìm được đường đi ngắn nhất tuyệt đối trong mọi cấu hình nhà kho phức tạp.
 - Hàm Manhattan: Đưa ra giá trị ước lượng ban đầu là $h = 4$. Do $4.0 > 3.0$, hàm Manhattan đã đánh giá cao hơn chi phí thực tế sinh ra từ các bước đi chéo. Việc vi phạm tính chất chấp nhận được này sẽ khiến giải thuật A^* không còn đảm bảo tìm được lời giải tối ưu khi chạy trên các lưới nhà kho quy mô lớn hoặc có vật cản phức tạp, có nguy cơ trả về một lộ trình không phải là ngắn nhất.

4 Task 4: Evacuation Route - Quy trình chuyển đổi từ bản đồ lưới sang ma trận trọng số và so sánh các hàm Heuristic qua một ví dụ

4.1 Quy trình chuyển đổi từ lưới sơ đồ sang ma trận trọng số

Để giải thuật A^* có thể duyệt trên cấu trúc đồ thị tổng quát, hệ thống thực hiện chuyển đổi sơ đồ phẳng floorPlan kích thước $m \times n$ sang ma trận kề weightMatrix theo quy trình hai bước cốt lõi:

1. **Tuyến tính hóa chỉ số:** Mỗi tọa độ hai chiều (r, c) trên lưới được ánh xạ thành một chỉ số một chiều duy nhất u phục vụ định vị đỉnh đồ thị theo công thức:

$$u = r \times n + c$$

2. **Xác lập liên kết lân cận và gán trọng số cạnh:** Với mỗi ô trống khả đạt (floorPlan[r][c] == 0), thuật toán tiến hành quét qua 8 ô xung quanh. Nếu ô lân cận nằm trong phạm vi lưới hợp lệ và không phải là vật cản, mỗi liên kết cạnh hướng $u \rightarrow v$ sẽ được thiết lập trên weightMatrix kèm theo trọng số phạt tương ứng:

- Gán trọng số bằng 1.0 cho 4 hướng di chuyển trực giao.
- Gán trọng số bằng 1.5 cho 4 hướng di chuyển chéo.

4.2 Ví dụ

Xét sơ đồ mặt bằng tòa nhà nhỏ kích thước 2×2 trống hoàn toàn ($m = 2, n = 2$). Ô xuất phát nằm tại góc trái trên $A(0, 0)$ (chỉ số phẳng là 0) và ô thoát hiểm là góc phải dưới $C(1, 1)$ (chỉ số phẳng là 3).

Áp dụng cơ chế tuyến tính hóa chỉ số, các ô lưới $(0, 0), (0, 1), (1, 0), (1, 1)$ tương ứng với các đỉnh 0, 1, 2, 3 trên đồ thị. Ma trận trọng số đầu ra cuối cùng kích thước 4×4 biểu diễn cấu trúc liên kết 8 hướng giữa các đỉnh được xác lập như sau:

$$\text{weightMatrix} = \begin{pmatrix} 0.0 & 1.0 & 1.0 & 1.5 \\ 1.0 & 0.0 & 1.5 & 1.0 \\ 1.0 & 1.5 & 0.0 & 1.0 \\ 1.5 & 1.0 & 1.0 & 0.0 \end{pmatrix}$$

Khoảng cách thực tế ngắn nhất là đường đi chéo trực tiếp từ đỉnh $0 \rightarrow 3$ với tổng chi phí thực tế là 1.5.

4.2.1 Hàm khoảng cách Manhattan

Khi cấu hình thuật toán sử dụng hàm ước lượng Manhattan, tiến trình tính toán và lưu vết các nhân trạng thái diễn ra như sau:

Bước duyệt	Node hiện tại	Open List (Node, g, h, f)	Closed List
Khởi tạo	-	$((0, 0), 0, 2, 2)$	$\emptyset$
Bước 1	$(0, 0)$	$((1, 1), 1.5, 0, 1.5), ((1, 0), 1, 1, 2), ((0, 1), 1, 1, 2)$	$\{(0, 0)\}$
Bước 2	$(1, 1)$	$((1, 0), 1, 1, 2), ((0, 1), 1, 1, 2)$ (Đạt đích)	$\{(0, 0), (1, 1)\}$

Nhận xét: Hàm Manhattan dự đoán khoảng cách ban đầu là $h = 2$. Khi loang tại Bước 1, ô đích $(1, 1)$ nhận giá trị thực tế $g = 1.5$ (do đi chéo) và $h = 0$, dẫn đến hàm đánh giá tổng $f = 1.5$. Vì đây là Node có chi phí f nhỏ nhất trong danh sách Open List, thuật toán lập tức bốc ô $(1, 1)$ ở Bước 2 và kết thúc tìm kiếm thành công.

4.2.2 Hàm khoảng cách Chebyshev

Khảo sát tiến trình duyệt trên cùng kích bản lưới 2×2 nhưng cấu hình thuật toán chạy với hàm Heuristic Chebyshev:

Bước duyệt	Node hiện tại	Open List (Node, g, h, f)	Closed List
Khởi tạo	-	$((0, 0), 0, 1, 1)$	$\emptyset$
Bước 1	$(0, 0)$	$((1, 1), 1.5, 0, 1.5), ((1, 0), 1, 1, 2), ((0, 1), 1, 1, 2)$	$\{(0, 0)\}$
Bước 2	$(1, 1)$	$((1, 0), 1, 1, 2), ((0, 1), 1, 1, 2)$ (Đã đạt đích)	$\{(0, 0), (1, 1)\}$

Nhận xét: Hàm Chebyshev đưa ra ước lượng ban đầu tại ô nguồn thấp hơn ($h = 1$). Tuy nhiên, ngay tại bước loang đầu tiên, chi phí tổng f của ô đích $(1, 1)$ vẫn hội tụ về đúng giá trị thực tế là 1.5. Thuật toán kết thúc chính xác sau 2 bước duyệt tương tự như hàm Manhattan.

4.3 So sánh và Đánh giá tổng quan

Mặc dù trong kích bản lưới kích thước nhỏ không vật cản này, cả hai hàm đều cho ra số bước duyệt Node tương đương và tìm được đường đi ngắn nhất, chúng ta vẫn thấy rõ sự khác biệt mang tính chất định hướng:

- **Về tính "chấp nhận được":** Bản chất chi phí thực tế của đường đi chéo là 1.5. Hàm Chebyshev ước lượng khoảng cách là 1 (≤ 1.5) nên luôn đảm bảo tính chấp nhận được. Hàm Manhattan ước lượng là 2 (> 1.5), tức là đã đánh giá cao hơn chi phí thực tế.
- **Hệ quả trong không gian lớn:** Vì Manhattan vi phạm tính chất chấp nhận được khi môi trường cho phép đi chéo với chi phí 1.5, nên trong các bản đồ mặt bằng thực tế quy mô lớn hoặc phức tạp hơn, việc dùng Manhattan có thể làm giải thuật A^* bị sai lệch hướng đi và không thể đảm bảo tìm được lộ trình thoát hiểm ngắn nhất tuyệt đối. Do đó, Chebyshev là sự lựa chọn an toàn và chính xác hơn cho bài toán này.